

RAJALAKSHMI ENGINEERING COLLEGE
(Autonomous)

RAJALAKSHMI NAGAR, THANDALAM, CHENNAI-602105

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**



**CP23221 ADVANCED SOFTWARE ENGINEERING
LABORATORY**

M.E. FIRST YEAR / SECOND SEMESTER



RAJALAKSHMI ENGINEERING COLLEGE

VISION & MISSION OF RAJALAKSHMI ENGINEERING COLLEGE

Vision of the Institute:

To be an institution of excellence in Engineering, Technology and Management Education & Research. To provide competent and ethical professionals with a concern for society.

Mission of the Institute:

M1: To impart quality technical education imbued with proficiency and humane values.

M2: To provide right ambience and opportunities for the students to develop into creative, talented and globally competent professionals.

M3: To promote research and development in technology and management for the benefit of the society.



RAJALAKSHMI ENGINEERING COLLEGE
B.E. COMPUTER SCIENCE AND ENGINEERING VISION:

To promote highly ethical and innovative computer professionals through excellence in teaching, training and research.

MISSION:

- To produce globally competent professionals, motivated to learn the emerging technologies and to be innovative in solving real world problems.
- To promote research activities amongst the students and the members of faculty that could benefit the society.
- To impart moral and ethical values in their profession.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: To equip students with essential background in computer science, basic electronics and applied mathematics.

PEO 2: To prepare students with fundamental knowledge in programming languages and tools and enable them to develop applications.

PEO 3: To encourage the research abilities and innovative project development in the field of networking, security, data mining, web technology, mobile communication and also emerging technologies for the cause of social benefit.

PEO 4: To develop professionally ethical individuals enhanced with analytical skills, communication skills and organizing ability to meet industry requirements.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate.

PSO 1: Foundation Skills: Ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, web design, machine learning, data analytics, and networking for efficient design of computer-based systems of varying complexity. Familiarity and practical competence with a broad range of programming language and open source platforms.

PSO 2: Problem-Solving Skills: Ability to apply mathematical methodologies to solve computational task, model real world problem using appropriate data structure and suitable algorithm. To understand the standard practices and strategies in software project development, using open- ended programming environments to deliver a quality product.

PSO 3: Successful Progression: Ability to apply knowledge in various domains to identify research gaps and to provide solution to new ideas, inculcate passion towards higher studies, creating innovative career paths to be an entrepreneur and evolve as an ethically social responsible computer science professional.

SYLLABUS

List of Experiments	
1	Write a Problem Statement to define a title of the project with bounded scope of project.
2	Select relevant process model to define activities and related task set for assigned project.
3	Prepare broad SRS (Software Requirement Specification) for the above selected projects.
4	Prepare Use Cases and Draw Use Case Diagram using modelling Tool.
5	Develop the activity diagram to represent flow from one activity to another for software development.
6	Develop data Designs using DFD, Decision Table or ER diagram.
7	Draw class diagram, sequence diagram, collaboration diagram, state transition diagram for the assigned project.
8	Write Test Cases to Validate requirements of assigned project from SRS Document.
9	Evaluate size of the project using function point metric for the assigned project.
10	Estimate cost of the project using COCOMO and COCOMO II for the assigned project.
11	Use CPM/PERT for scheduling the assigned project.
12	Use timeline Charts or Gantt Charts to track progress of the assigned project.

COURSE OUTCOMES (Cos)**Course Name: ADVANCED SOFTWARE ENGINEERING LABORATORY****Course Code: CP23221**

Outcome 1	Understand the requirements and use cases the client wants for the software being produced.
Outcome 2	Participate in drawing up the project plan including extent and work assessments, schedule, available resources, risk management, and architecture.
Outcome 3	Create and specify software design based on requirement specifications for implementation.
Outcome 4	Assess the extent and costs of a project using different assessment methods.
Outcome 5	Understand scheduling and tracking of projects using techniques.

CO-PO –PSO matrices of course

CO	PO1	PO2	PO3	PO4	PO5
CP23221.1	3	3	3	3	3
CP23221.2	2	3	3	3	2
CP23221.3	3	1	2	2	-
CP23221.4	2	3	1	2	-
CP23221.5	1	1	2	2	2
Average	2.2	2.2	2.2	2.4	2.3

Experiment 1

Write a Problem Statement to Define a Title of the Project with Bounded Scope

Title of the Project

Smart Emergency Blood & Donor Framework System

Introduction

In emergency medical situations, timely availability of blood is critical to save lives. Many hospitals and patients struggle to find suitable blood donors quickly due to lack of centralized communication and real-time donor availability tracking.

The proposed system aims to develop a web-based application using Python that connects blood donors, hospitals, and patients efficiently.

Problem Statement

Currently, there is no unified platform at the local level that allows hospitals and patients to quickly locate available blood donors based on blood group and geographical location. The manual process of contacting donors through phone calls or social media is time-consuming and unreliable during emergencies.

This project proposes the development of a **Smart Emergency Blood & Donor Management System** that:

- Registers blood donors with their details
- Allows hospitals to raise blood requests
- Matches donors based on blood group and location
- Displays available donors in real time
- Maintains request status and history

The system will improve response time and ensure efficient blood donation management.

Objectives

- To create a centralized blood donor database.
- To enable hospitals to post emergency blood requests.
- To automatically filter and match donors.
- To reduce time in finding compatible blood donors.
- To maintain secure and organized records.

Scope of the Project (Bounded Scope)

Development of a web-based application using Python (Flask).

Donor registration, login, and profile management.

Hospital registration and login functionality.

Creation and management of blood requests.

Automatic matching of donors based on:

- Blood group
- City
- Availability status

Admin panel to:

- View all donors and hospitals
- Monitor blood requests
- Update request status

Secure storage of data using MySQL database.

Basic search and filter functionality.

Simple dashboard showing:

- Total donors
- Active requests
- Completed requests

Technology to be Used

Frontend: HTML, CSS, Bootstrap

Backend: Python (Flask Framework)

Database: MySQL

Stakeholders

- Blood Donors
- Hospitals
- Patients
- System Administrator

Result:

The Smart Emergency Blood & Donor Management System aims to provide a simple, efficient, and reliable platform for managing emergency blood donation requests. The system focuses on solving a real-world problem with a bounded and clearly defined scope suitable for academic implementation in the Computer Science department.

Experiment 2

Select Relevant Process Model to Define Activities and Related Task Set for the Assigned Project

Selected Process Model

WATERFALL MODEL

Reason for Selecting Waterfall Model

The Waterfall Model is selected because:

1. The project requirements are clearly defined.
2. The system scope is well-bounded.
3. It is an academic project with fixed objectives.
4. Each phase can be completed before moving to the next phase.
5. It is easy to understand and suitable for documentation-based lab work.

Since the project does not require continuous requirement changes, the Waterfall Model is appropriate.

Phases of Waterfall Model Applied to the Project

Requirement Analysis

This is the first phase of the Waterfall Model.

- Identify users: Donor, Hospital, Admin
- Define functional requirements
- Define non-functional requirements
- Prepare SRS document

Output: Approved Requirement Specification

System Design

After requirements are finalized, system design begins.

- Design database tables (Donor, Hospital, Request)
- Create ER Diagram
- Create Use Case Diagram
- Design system architecture
- Design user interface

Output: Design Documents & Diagrams

Implementation

This phase converts design into actual code.

- Develop frontend using HTML, CSS
- Develop backend using Python (Flask)
- Connect MySQL database
- Implement donor matching logic

Output: Working Software Modules

Testing

In this phase, the developed system is tested.

- Test registration module
- Test login module
- Test blood request feature
- Verify matching accuracy
- Fix bugs

Output: Error-free validated system

Deployment

The tested system is deployed for use.

- Run project on local server
- Configure database
- Demonstrate to faculty

Output: Live Working System

Maintenance

After deployment, improvements and fixes are done.

- Correct minor errors
- Improve UI
- Optimize database queries
- Add small improvements if needed

Output: Updated & improved system

Key Characteristics of Waterfall Model

- Sequential flow
- No overlapping phases
- Documentation-heavy
- Easy to manage
- Best for fixed requirements

Result:

The Waterfall Model is suitable for the Smart Emergency Blood & Donor Management System because the project has clearly defined requirements and a fixed scope. Each development phase is completed before moving to the next, ensuring structured and systematic development.

Experiment 3

Software Requirement Specification (SRS)

Donor – A registered user willing to donate blood.

Hospital – An authorized medical center that raises blood requests.

Admin – The system administrator who manages users and requests.

Blood Request – An emergency request created by a hospital.

Product Perspective

The system is a standalone web application developed using HTML, CSS, Python (Flask), and MySQL. It follows a client-server architecture where users interact through a web browser.

Product Functions

The system performs the following functions:

- Registration of donors and hospitals
- User authentication (login and logout)
- Creation of blood requests by hospitals
- Matching donors based on blood group and city
- Updating donor availability
- Admin monitoring and management
- Updating request status

User Classes and Characteristics

Donor: A person with basic computer knowledge who can register, log in, and update availability.

Hospital: Authorized staff who can raise emergency blood requests.

Admin: A technical user with full system access who manages records and monitors system activities.

Operating Environment

The system operates on Windows or Linux operating systems using a web browser such as Chrome or Edge. Python 3.x, Flask framework, and MySQL database are required.

Design Constraints

The system is web-based only.

It operates at city level.

There is no GPS tracking feature.

There is no integration with government databases.

Assumptions and Dependencies

Users must have internet access.

Users must provide accurate information.

The database server must be properly configured.

Specific Requirements

Functional Requirements

- The system shall allow donors to register by providing name, blood group, city, contact details, and password.
- The system shall authenticate donors and hospitals using login credentials.
- The system shall allow hospitals to create emergency blood requests.
- The system shall filter donors based on blood group, city, and availability.
- The system shall allow donors to update availability status.
- The system shall allow admin to view, manage, and update user and request records.

Non-Functional Requirements

Performance- The system should display search results quickly without noticeable delay.

Security - User passwords should be stored securely.

Usability- The system should have a simple and user-friendly interface.

Reliability - The system should operate without frequent failures.

Maintainability - The system should be modular and easy to modify.

External Interface Requirements

User Interface: The system includes login page, registration page, dashboard, request creation page, and admin panel.

Hardware Interface: Computer system with internet connection.

Software Interface: Python, Flask framework, MySQL database, and web browser.

Result:

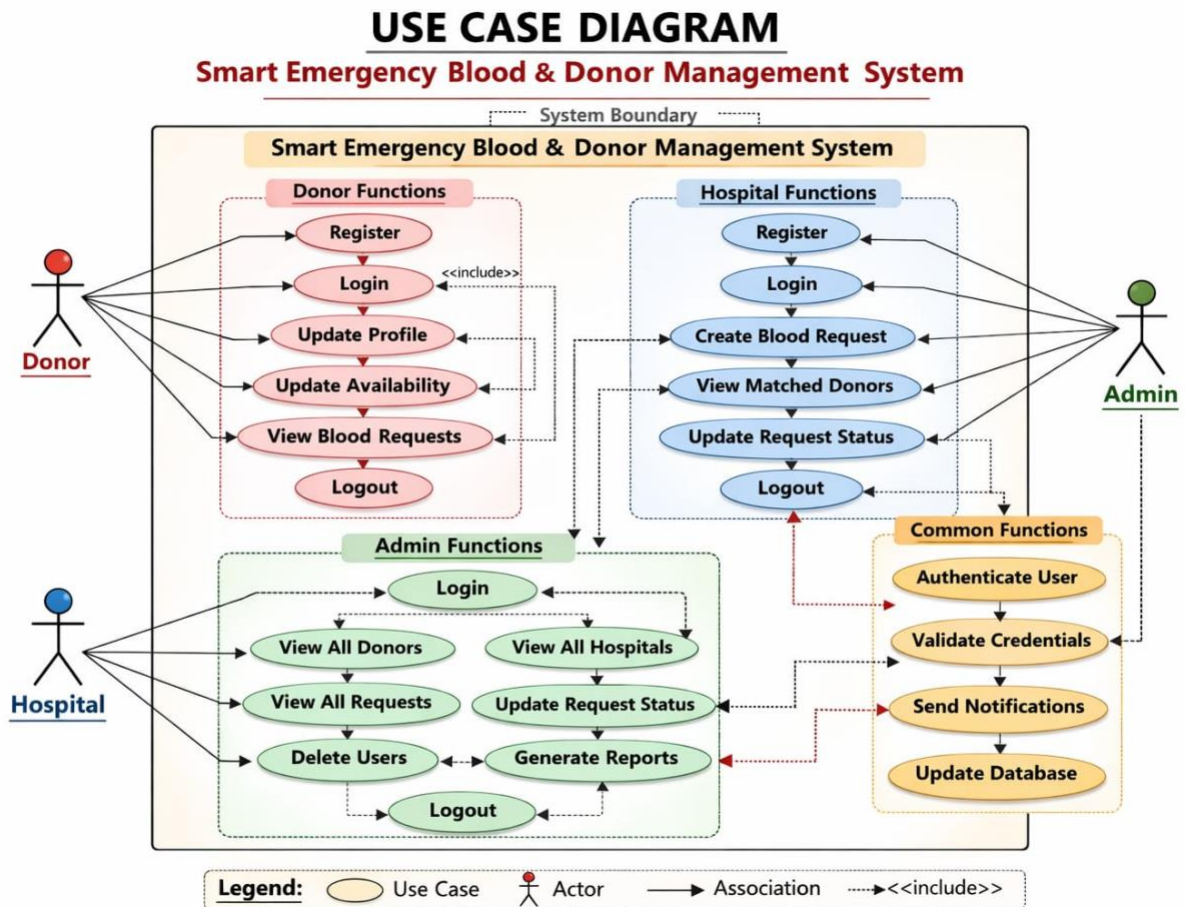
The Smart Emergency Blood & Donor Management System provides an organized and efficient way to manage blood donation requests. This SRS document clearly defines the system requirements and serves as a reference for design, development, and testing phases.

Experiment 4:

Prepare USE Cases and Draw Use Case Diagram using modelling Tool

A use case describes different user interactions with the system to achieve specific goals. A use case diagram visually represents the relationships between actors and system functionalities.

USE CASE DIAGRAM



ADMIN FUNCTIONS

- Login:** Admin enters username and password. System validates credentials. If correct, dashboard is displayed. If wrong, error message appears.
- Create / Update / Delete Function:** Admin can add new users (Doctor/Donor). Admin can modify existing user details. Admin can remove inactive users.
- Manage Settings:** Configure system settings. Set blood group categories. Manage notification settings. Maintain hospital information.
- System Management:** Monitor system activities. View reports. Maintain donor and request database.

DONOR FUNCTIONS

- Registration:** Donor fills registration form. Enters name, age, blood group, contact, address. System stores donor details in database.

- **Login:** Donor enters username and password. System authenticates donor. Access to donor dashboard is granted.
- **Update Donor Details:** Donor can edit phone number, address, availability. Updated data is saved in system.
- **View Donation Information & Eligibility:** Donor checks eligibility criteria. System shows last donation date. Displays whether donor is eligible to donate.
- **Make Request for Donation:** Donor can respond to emergency requests. Confirm availability for donation.
- **Search Donor:** Donor or patient can search by: Blood group, Location System displays matching donors.
- **Send Message:** Donor can send message to admin or patient. Used for communication or clarification.

USER / PATIENT FUNCTIONS

- **Login Page:** Patient logs into system. Access request features.
- **Search Donor:** Patient selects required blood group. System shows list of available donors.
- **Make Request for Donation:** Patient creates blood request. Enters hospital name, urgency level. System notifies matching donors.
- **Contact Us:** Patient sends query or complaint. Admin receives notification.

DOCTOR FUNCTIONS

- **Login to System:** Doctor enters login credentials. System verifies and grants access.
- **Open Patient Detail Form:** View patient medical details. Check blood requirement information.
- **Do Check-Up:** Verify donor health condition. Confirm donor eligibility.
- **Update Status of Donor:** Mark donor as: Eligible, not eligible, temporarily unavailable

Common Included Function (<<include>>)

Many use cases include Login because:

- User must authenticate before accessing features.
- It ensures security and data protection.

Result:

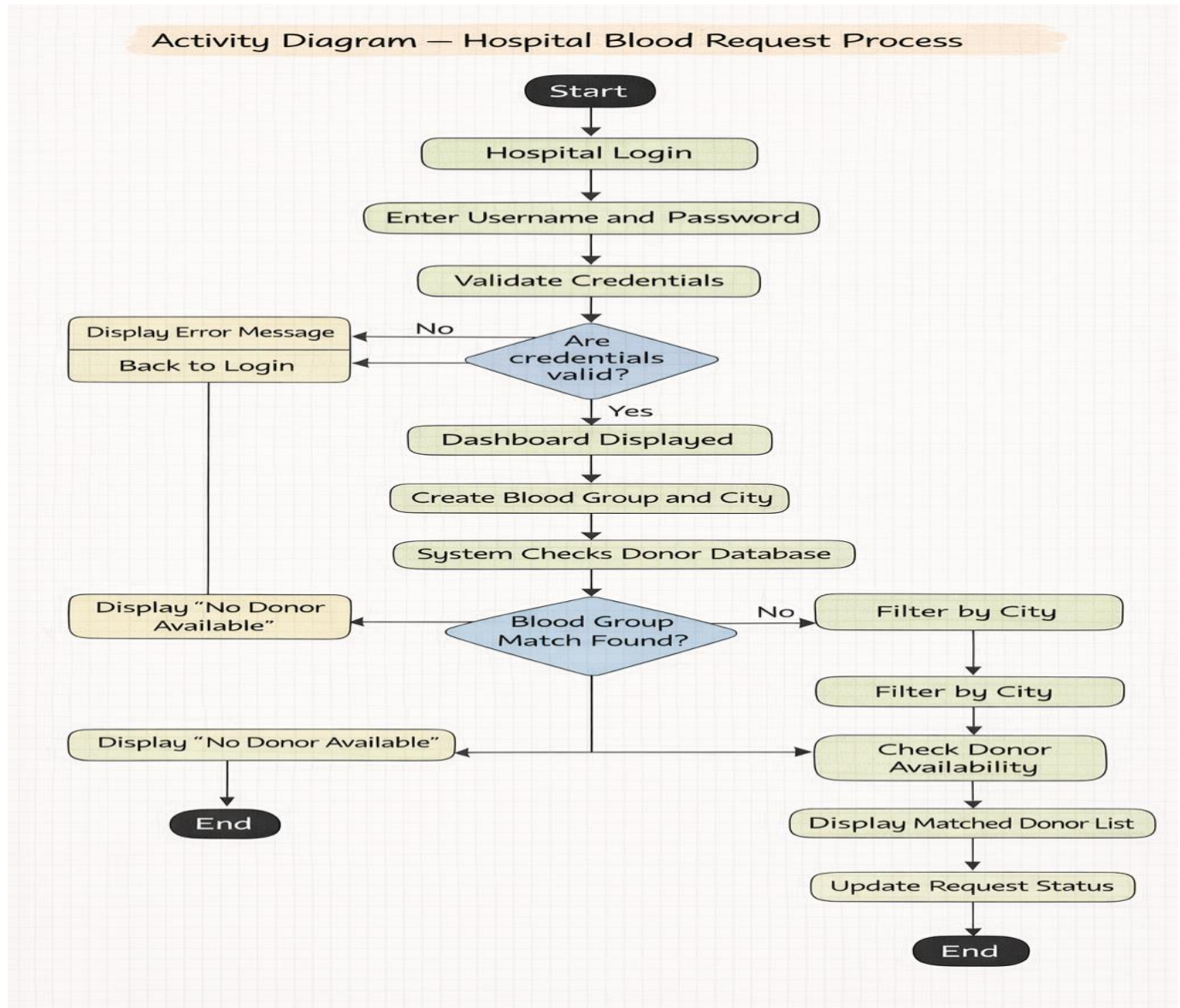
The SRS document Clearly define the system required and serve as a referenced design development and testing phase.

Experiment 5

Develop the Activity Diagram to Represent Flow from One Activity to Another

An Activity Diagram represents the workflow of a system. It shows the sequence of activities, decision points, and flow of control from start to end. In this project, the Activity Diagram explains the process of handling a blood request from hospital login to displaying matched donors.

Activity Diagram



Result:

Thus, the process shows the sequence of activities, decision points, and flow of control from start to end successfully.

Experiment 6

Develop ER Diagram for Blood & Donor Management System and Data Flow Diagram (DFD) – Blood & Donor Management System

Aim

To design an **Entity Relationship (ER) Diagram** for the Blood & Donor Management System and identify entities, attributes, and relationships.

Entities and Attributes

Donor

Attributes:

- Donor_ID (Primary Key)
- Name
- Age
- Gender
- Blood_Group
- Contact_Number
- Address
- Last_Donation_Date
- Availability_Status

Description:

Stores complete details of blood donors registered in the system.

Patient

Attributes:

- Patient_ID (Primary Key)
- Name
- Age
- Gender
- Blood_Group_Required
- Contact_Number

- Hospital_Name
- Address

Description:

Contains details of patients who request blood.

Doctor

Attributes:

- Doctor_ID (Primary Key)
- Name
- Qualification
- Contact_Number
- Hospital_Name

Description:

Doctor verifies donor eligibility and updates health status.

Admin

Attributes:

- Admin_ID (Primary Key)
- Name
- Email
- Contact_Number
- Username
- Password

Description:

Admin manages the entire system.

Blood_Request

Attributes:

- Request_ID (Primary Key)
- Patient_ID (Foreign Key)
- Blood_Group
- Quantity
- Request_Date

- Status

Description:

Stores all blood requests raised by patients.

Donation**Attributes:**

- Donation_ID (Primary Key)
- Donor_ID (Foreign Key)
- Request_ID (Foreign Key)
- Donation_Date
- Donation_Status

Description:

Records successful blood donations.

Relationships**1. Patient — Creates — Blood_Request**

- One patient can create many requests.
- One request belongs to one patient.
(Relationship: One-to-Many)

2. Donor — Donates — Donation

- One donor can donate many times.
- Each donation belongs to one donor.
(Relationship: One-to-Many)

3. Blood_Request — Fulfilled_By — Donation

- One request can be fulfilled by one or more donations.
(Relationship: One-to-Many)

4. Doctor — Verifies — Donor

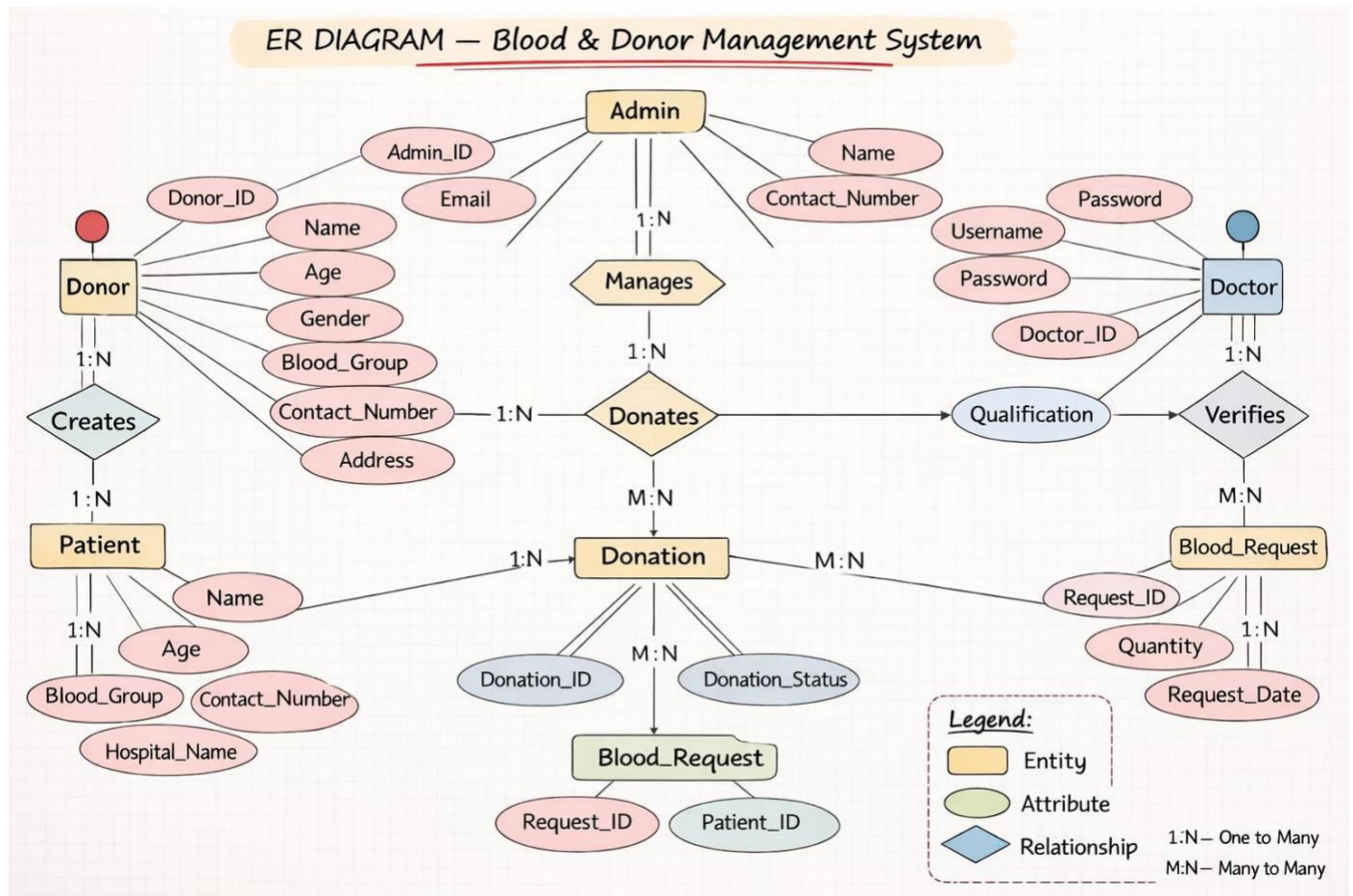
- One doctor verifies many donors.
(Relationship: One-to-Many)

5. Admin — Manages — System

- Admin manages donors, doctors, and requests.

Steps to Draw ER Diagram in Record

1. Draw rectangles for Entities.
2. Draw ovals for Attributes.
3. Underline Primary Keys.
4. Draw diamonds for Relationships.
5. Connect entities with relationships using lines.
6. Mention cardinality (1:N, M:N).



Data Flow Diagram (DFD) – Blood & Donor Management System

Level 0 DFD (Context Level)

Process Table

Process No	Process Name	Input	Output	External Entity
0	Blood & Donor Management System	Registration data, Login details, Blood request	Confirmation, Donor list, Request status	Donor, Patient, Admin, Doctor

Explanation

- The entire system is shown as a single process.
- Donor sends registration and login details.
- Patient sends blood request.
- Doctor updates donor status.
- Admin manages system data.
- System returns confirmation, donor list, and request updates.

Level 1 DFD

Level 1 breaks the main process into sub-processes.

Process Table

Process No	Process Name	Input	Output	Data Store Used
1.0	Manage Registration	Donor details	Registration confirmation	Donor DB
2.0	Manage Login	Username, Password	Login status	User DB
3.0	Search Donor	Blood group, Location	List of matching donors	Donor DB
4.0	Create Blood Request	Patient details, Blood group	Request ID, Status	Request DB
5.0	Verify Donor	Donor health details	Eligibility status	Donor DB
6.0	Update Request Status	Request ID, Status update	Updated request information	Request DB

Data Store Table

Data Store	Description
D1 – Donor DB	Stores donor details
D2 – Patient DB	Stores patient information
D3 – Request DB	Stores blood request records
D4 – Admin DB	Stores admin login data

External Entities

Entity	Role
Donor	Registers and donates blood
Patient	Requests blood
Doctor	Verifies donor eligibility
Admin	Manages the system

Explanation of Data Flow

1. Donor registers → Data stored in Donor DB.
2. Patient creates blood request → Stored in Request DB.
3. System searches donor based on blood group.
4. Doctor verifies donor health.
5. Admin monitors and updates system data.
6. System sends confirmation and status updates.

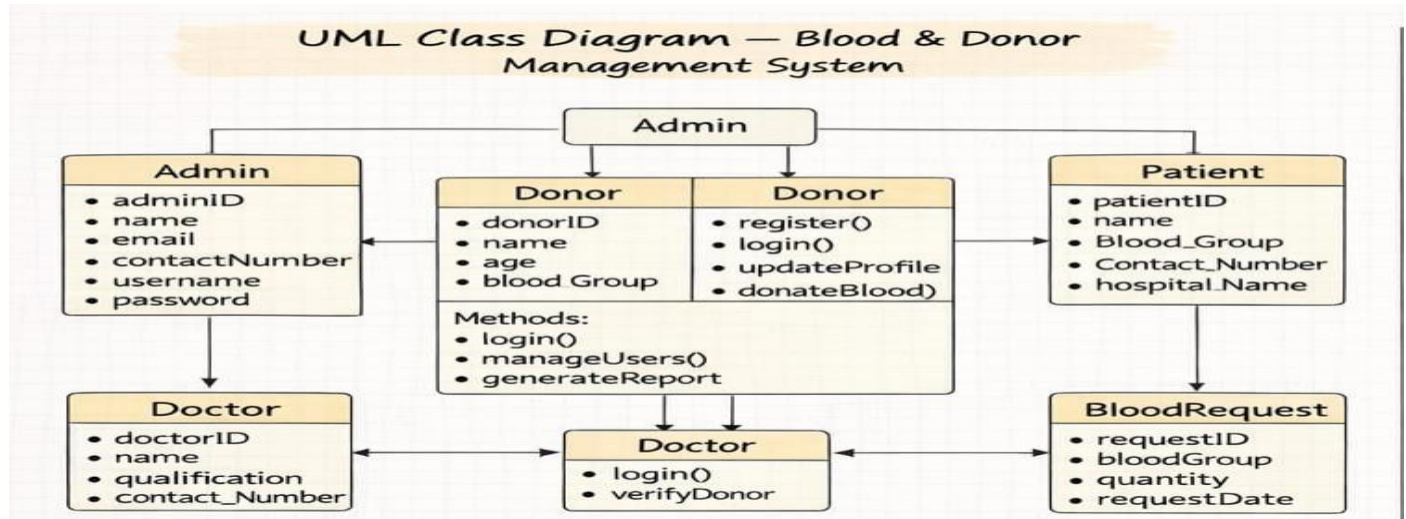
Result:

The ER Diagram and DFD table clearly represents the database structure of the Blood & Donor Management System by defining entities, attributes, and relationships. It helps in database design and implementation.

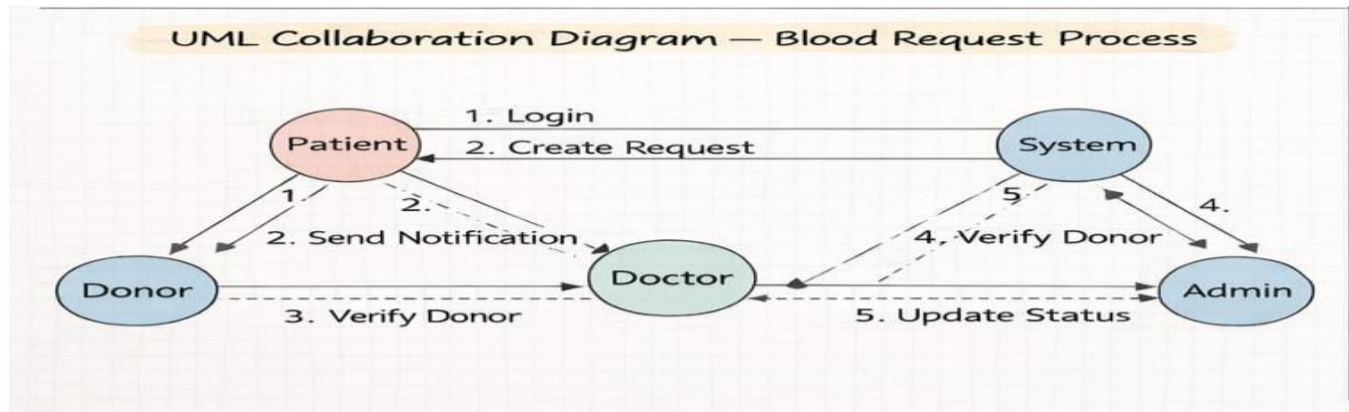
Experiment: 07

Draw class diagram, sequence diagram, Collaboration Diagram, State Transition Diagram for the assigned project and give the explanation

UML Diagram

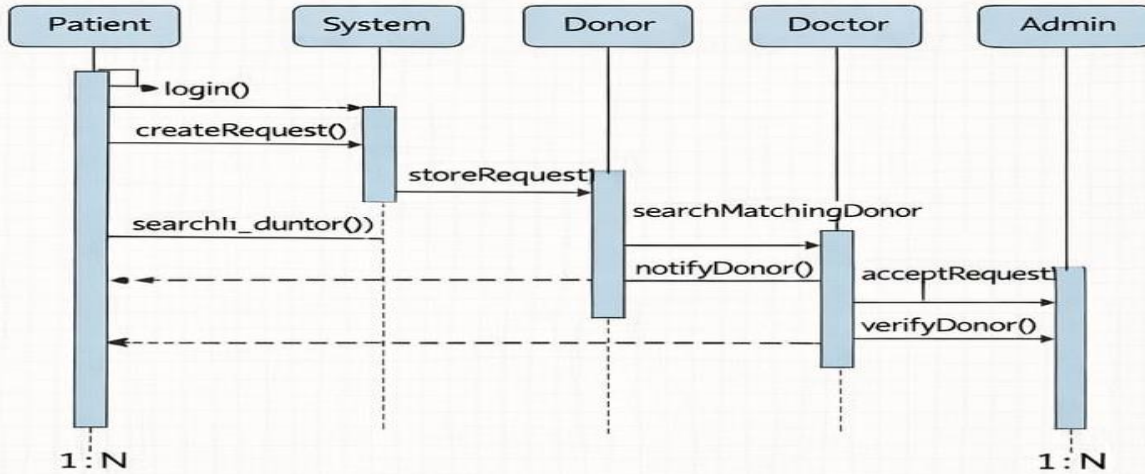


Collaboration Diagram blood request Process



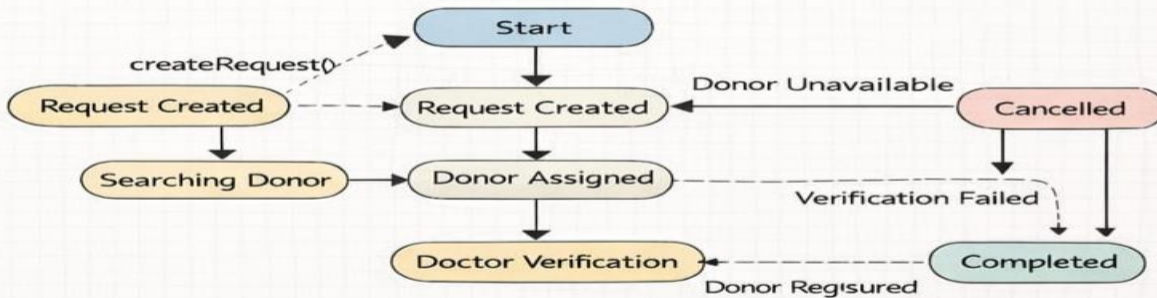
Sequence Diagram

UML Sequence Diagram – Blood Request Process



State Transition Diagram

UML State Transition Diagram – Blood Request Status



Result:

Class Diagram shows structure. Sequence Diagram shows time order interaction. Collaboration Diagram shows communication flow. State Diagram shows status changes.

Experiment: 08

Test Cases to Validate Requirements

Test Case Format

Test Case ID	Requirement	Test Description	Input	Expected Output	Status
--------------	-------------	------------------	-------	-----------------	--------

Registration Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_01	User Registration	Check donor registration with valid data	Valid name, age, blood group, contact	Registration successful message	Pass
TC_02	User Registration	Check registration with missing fields	Blank name	Error message displayed	Pass
TC_03	User Registration	Check invalid age input	Age < 18	"Not eligible" message	Pass

Login Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_04	Login	Valid login credentials	Correct username & password	Dashboard displayed	Pass
TC_05	Login	Invalid password	Correct username, wrong password	Error message	Pass
TC_06	Login	Empty fields	Blank username/password	Validation message	Pass

Donor Search Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_07	Search Donor	Search by blood group	O+	List of O+ donors displayed	Pass
TC_08	Search Donor	Search with unavailable blood group	Rare group	"No donor found" message	Pass

Blood Request Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_09	Create Request	Create new blood request	Valid patient details	Request ID generated	Pass
TC_10	Update Status	Update request status	Request ID	Status updated successfully	Pass

Doctor Verification Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_11	Verify Donor	Doctor verifies donor eligibility	Valid donor ID	Donor marked eligible	Pass
TC_12	Verify Donor	Reject unhealthy donor	Health condition = Not fit	Donor marked not eligible	Pass

Admin Module

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_13	Manage Users	Delete donor account	Valid Donor ID	Donor removed from system	Pass
TC_14	Generate Report	Generate donation report	Date range	Report displayed	Pass

Database Validation

TC_ID	Requirement	Test Description	Input	Expected Output	Status
TC_15	Data Storage	Check donor data stored	Register donor	Data saved in database	Pass
TC_16	Data Integrity	Check foreign key constraint	Invalid Patient ID	Error message	Pass

Result:

The above test cases validate: Functional requirements, Input validation, Security requirements, Database integrity, System workflow

Experiment:09

Evaluate Project Size Using Function Point (FP) Metric

Step 1: Identify and Classify Components

Function Point Analysis is based on:

1. **External Inputs (EI)**
2. **External Outputs (EO)**
3. **External Inquiries (EQ)**
4. **Internal Logical Files (ILF)**
5. **External Interface Files (EIF)**

Step 2: Count the Components

External Inputs (EI)

User inputs that update internal data.

Function	Count
Donor Registration	1
Patient Registration	1
Doctor Registration	1
Login	1
Create Blood Request	1
Update Donor Profile	1
Update Request Status	1

Total EI = 7

Assume Average Complexity → Weight = 4

FP (EI) = $7 \times 4 = 28$

External Outputs (EO)

System-generated outputs with processing logic.

Function	Count
Donation Report	1
Blood Request Status	1
Donor List	1

Total EO = 3

Assume Average Complexity → Weight = 5

$$FP(EO) = 3 \times 5 = 15$$

External Inquiries (EQ)

Simple retrieval without complex processing.

Function	Count
Search Donor	1
View Request Details	1

Total EQ = 2

Assume Average Complexity $\rightarrow$ Weight = 4

$$FP(EQ) = 2 \times 4 = 8$$

Internal Logical Files (ILF)

Internal database tables.

File	Count
Donor Table	1
Patient Table	1
Doctor Table	1
Blood Request Table	1
Donation Table	1
Admin Table	1

Total ILF = 6

Assume Average Complexity $\rightarrow$ Weight = 10

$$FP(ILF) = 6 \times 10 = 60$$

External Interface Files (EIF)

External system interfaces (assume none).

Total EIF = 0

Step 3: Calculate Unadjusted Function Points (UFP)

$$\begin{aligned}
 UFP &= EI + EO + EQ + ILF + EIF \\
 UFP &= 28 + 15 + 8 + 60 + 0 \\
 UFP &= 111
 \end{aligned}$$

Step 4: Calculate Value Adjustment Factor (VAF)

There are 14 General System Characteristics (GSCs).

Assume average rating of 3 for each.

$$Total\ GSC = 14 \times 3 = 42$$

$$\begin{aligned} VAF &= 0.65 + (0.01 \times 42) \\ VAF &= 0.65 + 0.42 \\ VAF &= 1.07 \end{aligned}$$

Step 5: Calculate Adjusted Function Points (AFP)

$$\begin{aligned} AFP &= UFP \times VAF \\ AFP &= 111 \times 1.07 \\ AFP &= 118.77 \\ AFP &\approx 119 \end{aligned}$$

Result

The estimated project size using Function Point Metric is: **119 Function Points**

Experiment: 10

Cost Estimation Using COCOMO & COCOMO II

Adjusted Function Points (AFP) = **119 FP**

Assume:

1 Function Point $\approx$ **50 Lines of Code (LOC)** (for web-based system)

$$LOC = 119 \times 50 = 5950 \approx 6KLOC$$

So, Project Size $\approx$ **6 KLOC**

Basic COCOMO Model

Formula:

$$Effort(PM) = a \times (KLOC)^b$$

For **Organic Mode** (small-medium project):

- $a = 2.4$
- $b = 1.05$

Effort Calculation

$$\begin{aligned} Effort &= 2.4 \times (6)^{1.05} \\ 6^{1.05} &\approx 6.35 \\ Effort &= 2.4 \times 6.35 \\ Effort &\approx 15.24 \text{ Person} - \text{Months} \end{aligned}$$

Development Time

$$\begin{aligned} Time &= 2.5 \times (Effort)^{0.38} \\ Time &= 2.5 \times (15.24)^{0.38} \\ (15.24)^{0.38} &\approx 2.82 \\ Time &\approx 2.5 \times 2.82 \\ Time &\approx 7.05 \text{ Months} \end{aligned}$$

Team Size

$$\begin{aligned} TeamSize &= Effort/Time \\ &= 15.24/7.05 \\ &\approx 2 \text{ Persons} \end{aligned}$$

Cost Estimation

Assume salary per person per month = ₹40,000

$$\begin{aligned} TotalCost &= Effort \times Salary \\ &= 15.24 \times 40,000 \\ &= ₹6,09,600 \end{aligned}$$

COCOMO Result

Effort = **15 Person-Months**
Time = **7 Months**
Team Size = **2 Members**
Estimated Cost $\approx$ **₹6 Lakhs**

COCOMO II Model

Formula:

$$Effort = 2.94 \times (KLOC)^E$$

Assume average scale factor $\rightarrow E \approx 1.10$

$$\begin{aligned} Effort &= 2.94 \times (6)^{1.10} \\ 6^{1.10} &\approx 6.86 \\ Effort &= 2.94 \times 6.86 \\ Effort &\approx 20.15 \text{ Person} - \text{Months} \end{aligned}$$

Development Time (COCOMO II)

$$\begin{aligned} Time &= 3.67 \times (Effort)^{0.28} \\ Time &= 3.67 \times (20.15)^{0.28} \\ (20.15)^{0.28} &\approx 2.32 \\ Time &\approx 3.67 \times 2.32 \\ Time &\approx 8.51 \text{ Months} \end{aligned}$$

Cost (COCOMO II)

$$\begin{aligned} Cost &= 20.15 \times 40,000 \\ &= ₹8,06,000 \end{aligned}$$

COCOMO II Result

Effort = **20 Person-Months**
Time = **8.5 Months**
Estimated Cost $\approx$ **₹8 Lakhs**

Result:

Model	Effort (PM)	Time (Months)	Estimated Cost
COCOMO	15	7	₹6 Lakhs
COCOMO II	20	8.5	₹8 Lakhs

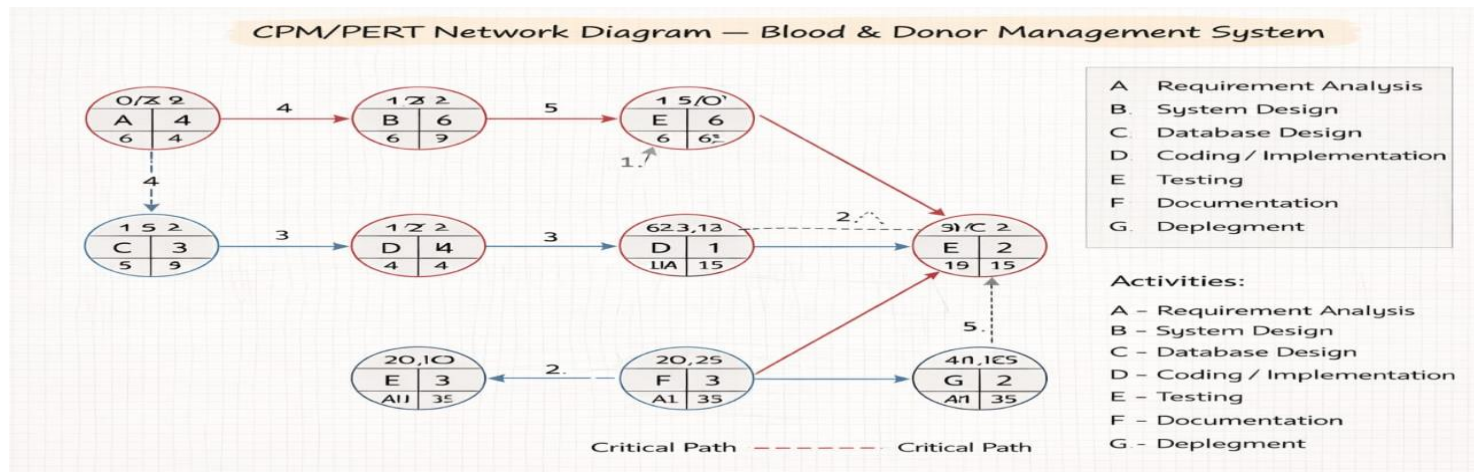
Experiment: 11

Use CPM/PERT for Scheduling the Project

1. Identify Project Activities

Activity	Description	Predecessor	Duration (Days)
A	Requirement Analysis	—	4
B	System Design	A	5
C	Database Design	B	3
D	Coding / Implementation	C	8
E	Testing	D	4
F	Documentation	D	3
G	Deployment	E, F	2

2. CPM Network Flow



3. Forward Pass (Earliest Time)

Activity	Duration	ES	EF
A	4	0	4
B	5	4	9
C	3	9	12
D	8	12	20
E	4	20	24
F	3	20	23
G	2	24	26

Project Completion Time = **26 Days**

4. Backward Pass (Latest Time)

Activity	Duration	LS	LF
G	2	24	26
E	4	20	24
F	3	21	24
D	8	12	20
C	3	9	12
B	5	4	9
A	4	0	4

5. Slack Time

Activity	Slack
A	0
B	0
C	0
D	0
E	0
F	1
G	0

6. Critical Path

Critical Path = **A → B → C → D → E → G**

Total Project Duration = **26 Days**

Activities on the critical path have **zero slack**.

7. PERT Time Estimation

Formula:

$$TE = (O + 4M + P)/6$$

Where

O = Optimistic time=6

M = Most likely time=8

P = Pessimistic time=10

Example (Activity D – Coding):

$$TE = (6 + 4 \times 8 + 10)/6$$

$$TE = 48/6 = 8 \text{ days}$$

Result:

Using CPM and PERT techniques, the project schedule was developed and the **critical path was identified**. The total estimated completion time of the Blood & Donor Management System project is **26 days**.

Experiment 12

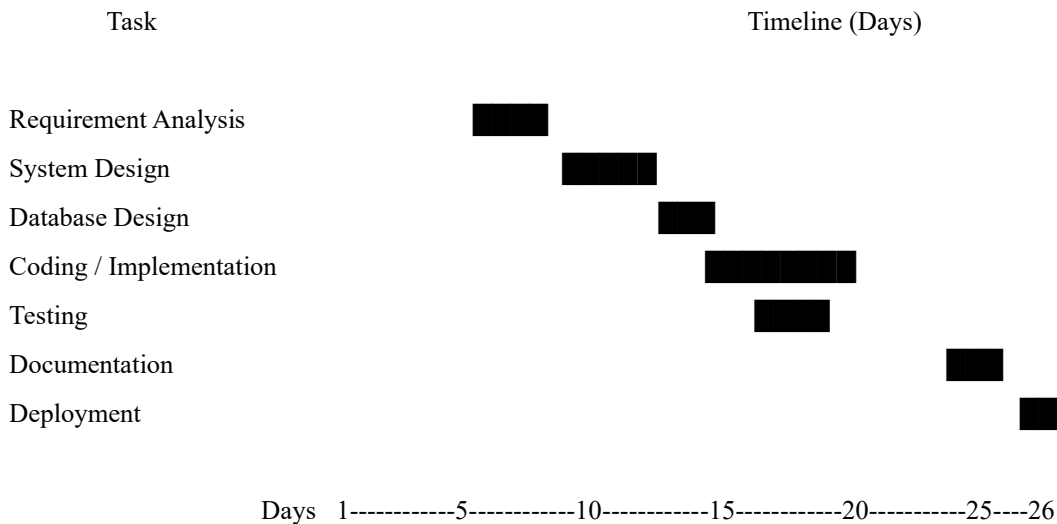
Use Timeline Chart / Gantt Chart to Track Project Progress

A **Gantt Chart** is a visual timeline used to track project activities, start time, duration, and completion status.

Project Activities Timeline

Task	Activity	Start Day	Duration (Days)	End Day
A	Requirement Analysis	Day 1	4	Day 4
B	System Design	Day 5	5	Day 9
C	Database Design	Day 10	3	Day 12
D	Coding / Implementation	Day 13	8	Day 20
E	Testing	Day 21	4	Day 24
F	Documentation	Day 21	3	Day 23
G	Deployment	Day 25	2	Day 26

Gantt Chart (Timeline Representation)



Explanation of Each Activity

Requirement Analysis: Understanding system requirements such as donor registration, blood search, and request management.

System Design: Designing system architecture, modules, and workflow of the Blood & Donor Management System.

Database Design: Creating database tables such as **Donor, Patient, Blood Request, Admin**.

Coding / Implementation: Developing the system using programming language (Python) and implementing system functionalities.

Testing: Checking the system for errors and verifying that all modules work correctly.

Documentation: Preparing system documentation, user manuals, and project reports.

Deployment: Final installation of the system so users (Admin, Donor, Patient) can access it.

Result:

The **Gantt Chart** helps track project progress, manage time effectively, and ensure that the Blood & Donor Management System is completed within the planned schedule.